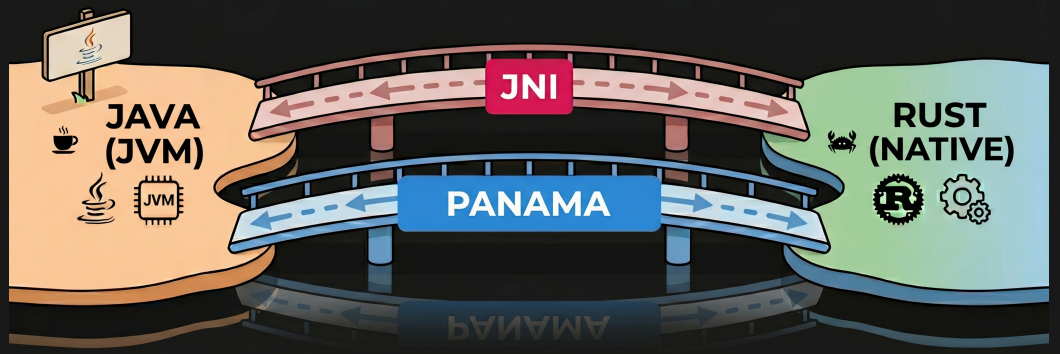
gortizja.com

Native Speed, Java Comfort

Calling Rust from the JVM with Project Panama

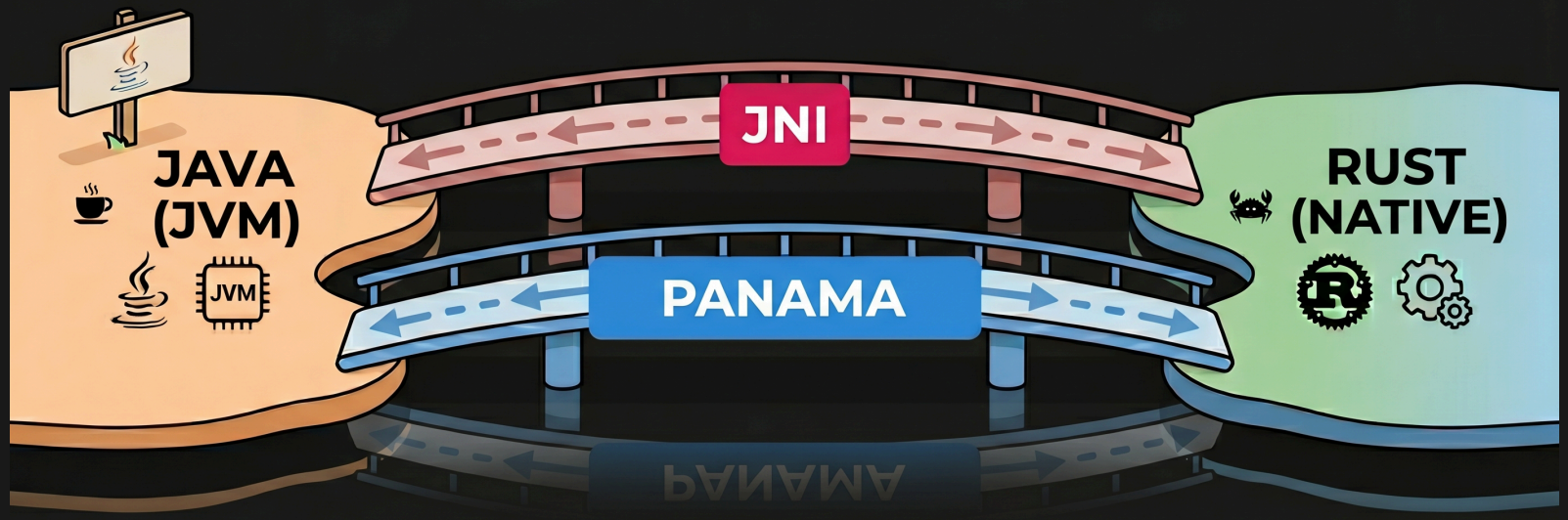


Gonzalo Ortiz · Apache Pinot PMC · StarTree

Why call native code?

- **Ecosystem access** — reuse Rust/C/C++ libraries that already exist.
  ML inference (ONNX), compression (zstd), databases. Don't rewrite them.
- **Ergonomics** — some problems are more natural with direct memory control.
  SIMD intrinsics, memory-mapped structures, zero-copy I/O.
- **Performance** — bypass JIT/GC limits…
  Be careful — the JVM is surprisingly good. Native isn't a magic speed button.

The two bridges



JNI — in the JDK since Java 1.1 · **Panama FFM** — finalized in Java 22

Same call, two philosophies

JNI — object-centric

```toml
[lib]
crate-type = ["cdylib"]

[dependencies]
jni = "0.21"
```

- Rust receives Java objects
- JVM may pin objects
- Rust crate coupled to the JVM

Panama — mem-centric

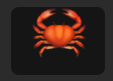
```toml
[lib]
crate-type = ["cdylib"]
```

- Rust receives raw memory
- No GC interaction — data lives off-heap
- Same standard C ABI
  - Already used by Python, Go, Node…

add(a, b)

JNI

```rust
#[no_mangle]
pub extern "system"
fn Java_com_example_Lib_add(
    _env: JNIEnv, // Used to access the JVM
    _class: JClass, // Our "Lib" class
    a: jint,
    b: jint,
) -> jint {
    add(a, b)
}
```



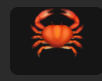
```java
public class Lib {
    static { System.loadLibrary("mylib"); }
    public static native int add(int a, int b);
}

int r = Lib.add(2, 3);   // → 5
```



Panama

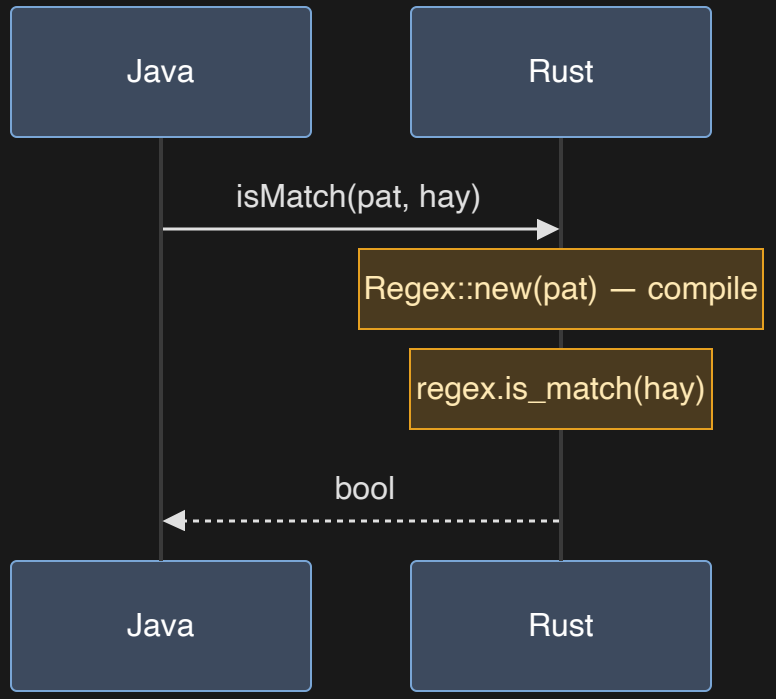
```rust
#[no_mangle]
pub extern "C"
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```



```java
private static final MethodHandle ADD;
static {
    SymbolLookup lib = SymbolLookup.libraryLookup(
        "libmylib.so", Arena.global());
    ADD = Linker.nativeLinker().downcallHandle(
        lib.find("add").orElseThrow(),
        FunctionDescriptor.of(
            JAVA_INT, JAVA_INT, JAVA_INT));
}
int r = (int) ADD.invoke(2, 3);   // → 5
```



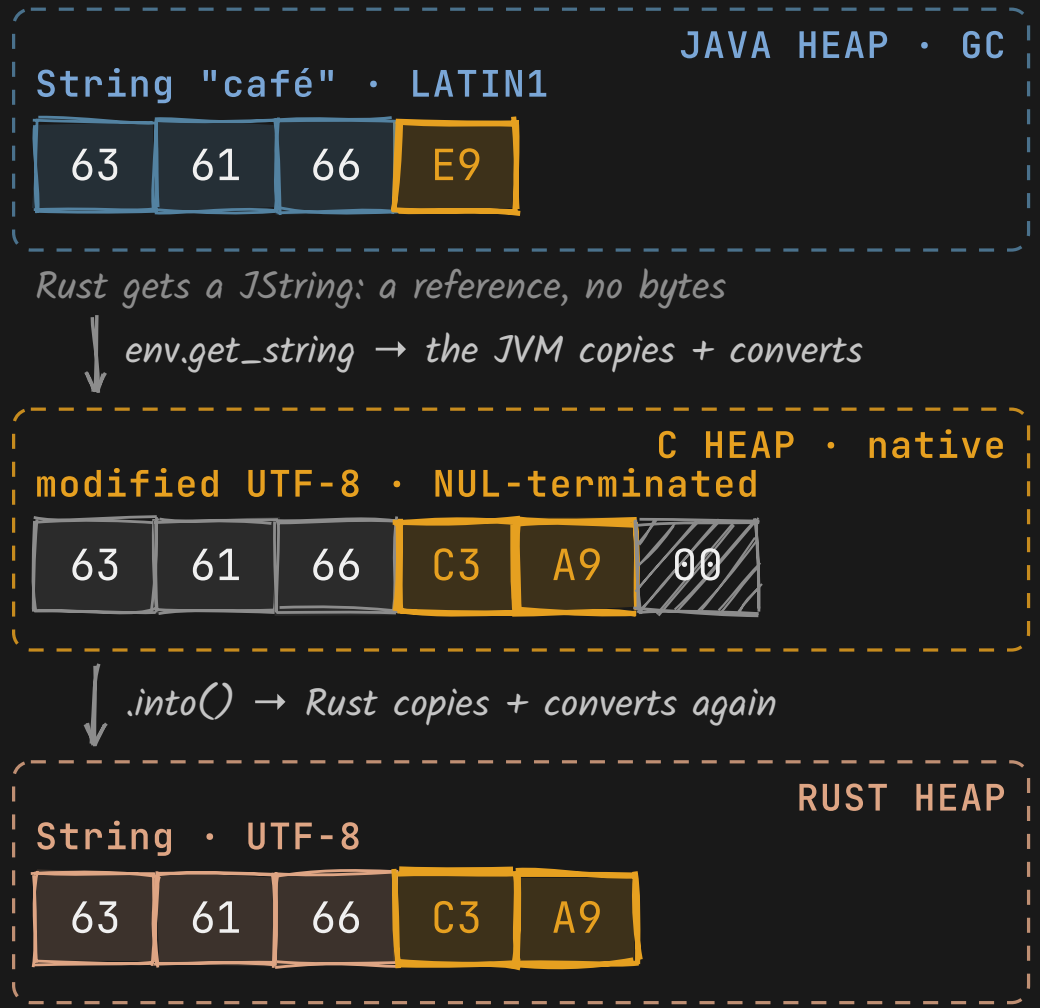
Using strings



`isMatch(pat, hay)` — JNI

```java
static native boolean isMatch(String pat, String hay);
boolean m = JniRsRegex.isMatch("café", "un café");
```

```rust
#[no_mangle]
pub extern "system"
fn Java_org_example_rust_JniRsRegex_isMatch(
    mut env: JNIEnv,
    _class: JClass,
    pattern: JString, // pointer to Java String
    haystack: JString, // pointer to Java String
) -> jboolean {
    let pat: String = env.get_string(&pattern)
      .unwrap().into();
    let hay: String = env.get_string(&haystack)
      .unwrap().into();
    Regex::new(&pat).unwrap().is_match(&hay)
      as jboolean
}
```



COPIES

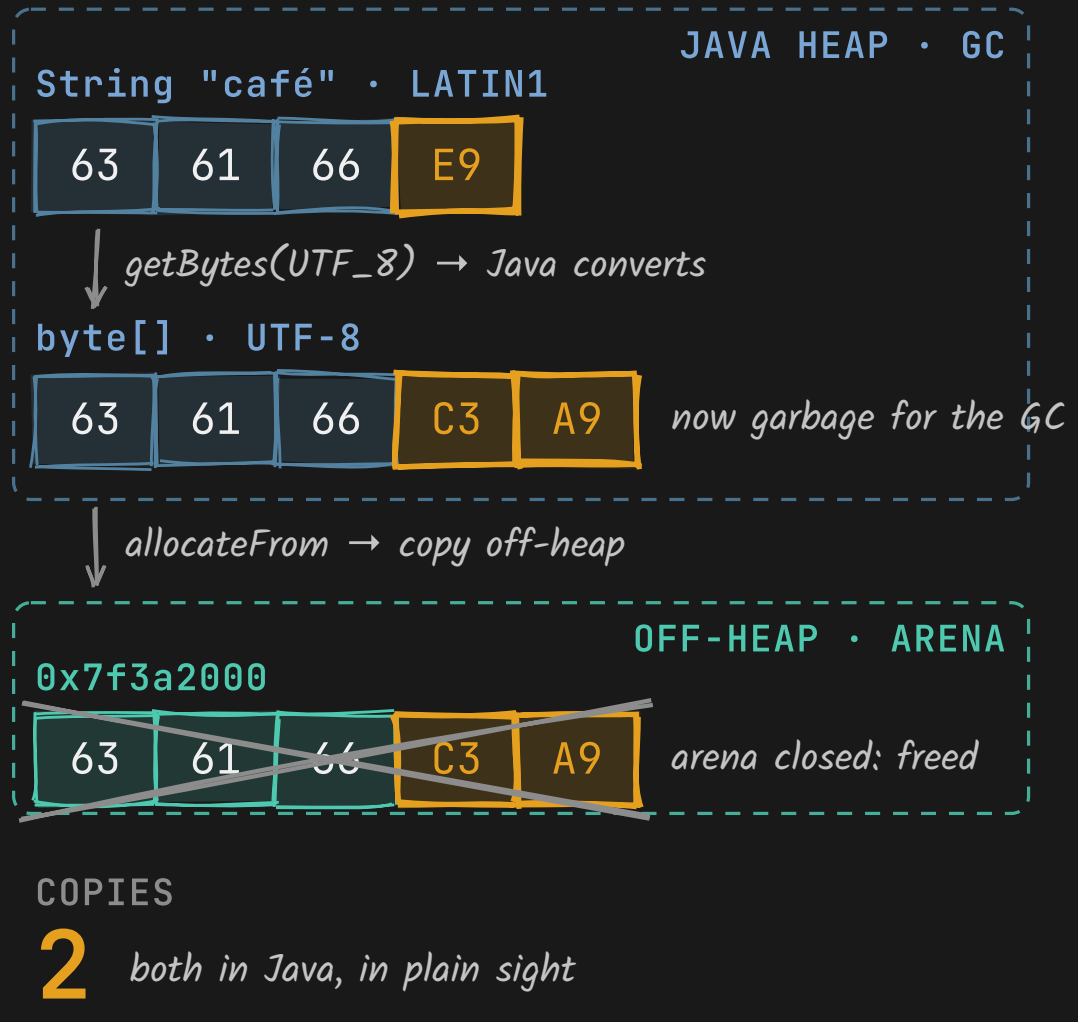
2 *both inside Rust, through the JVM*

`isMatch(pat, hay)` — Panama

```java
static final MethodHandle IS_MATCH = Linker
        .libraryLookup(...)
        .downcallHandle(...);

public boolean isMatch(String pat, String hay) {
    try (Arena arena = Arena.ofConfined()) {
        MemorySegment cP = arena.allocateFrom(
            JAVA_BYTE,
            pat.getBytes(UTF_8));
        MemorySegment cH = arena.allocateFrom(
            JAVA_BYTE,
            hay.getBytes(UTF_8));
        return (boolean) IS_MATCH.invoke(
            cP, cP.byteSize(), cH, cH.byteSize());
    }
}
```

```rust
// Rust: a view on the arena bytes, no copy
let pat = unsafe { std::str::from_utf8_unchecked(
    std::slice::from_raw_parts(pat_ptr, pat_len)) };
```

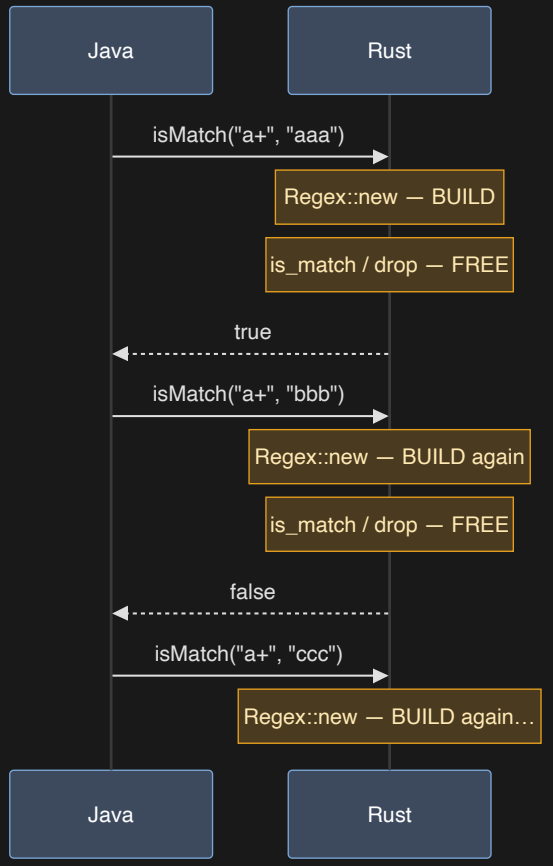


COPIES

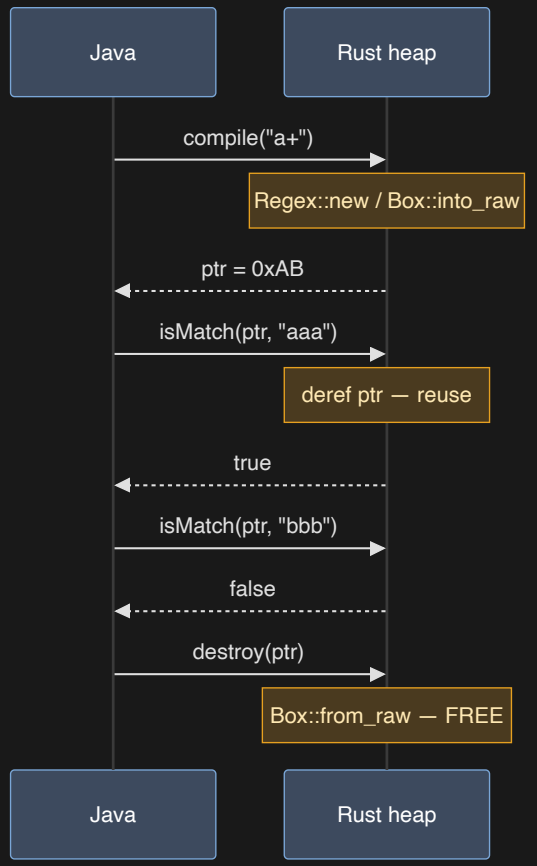
2 both in Java, in plain sight

Compile every call → compile once

✗ Problem: rebuild on every call



✓ Solution: compile once, pass a handle



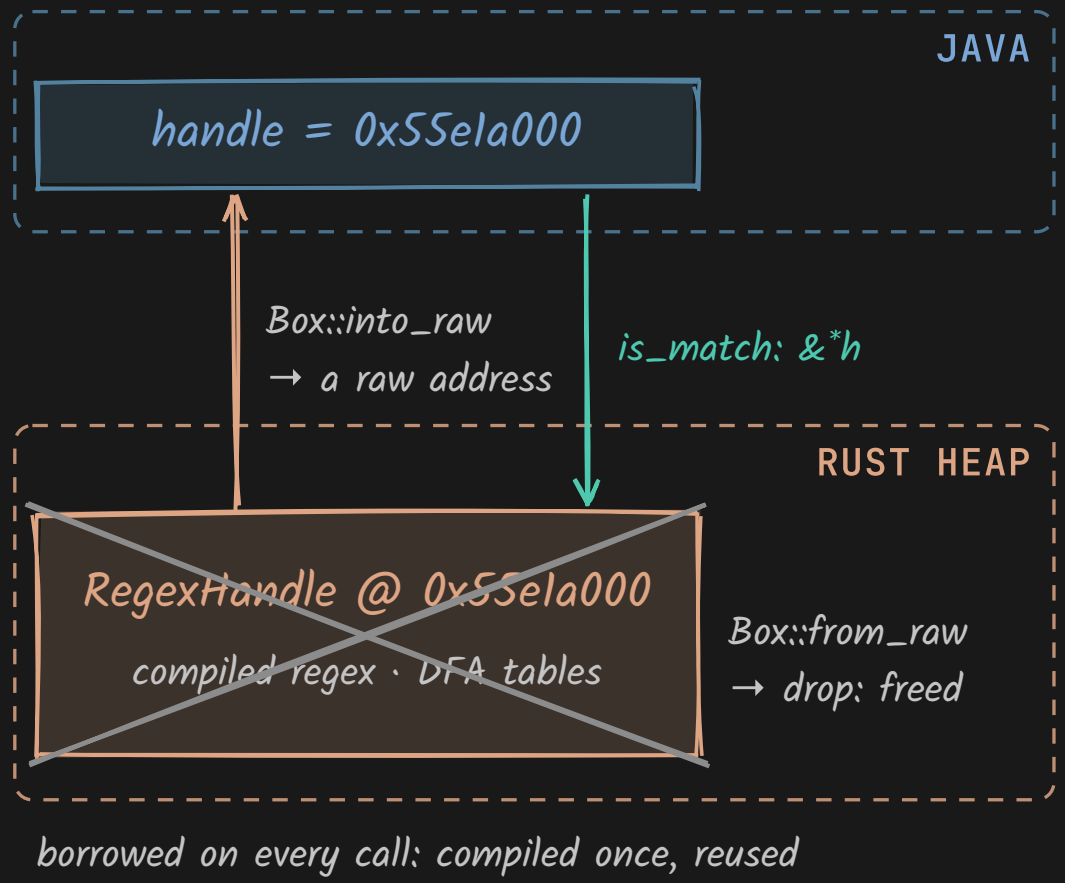
Compiled Regex — Rust

```rust
pub struct RegexHandle { regex: Regex }

// create → Java gets a raw ptr it owns
#[no_mangle]
pub extern "C" fn create(
    pat: *const u8, len: usize,
) -> *mut RegexHandle {
    let re = Regex::new(
        bytes_to_str(pat, len)).unwrap();
    Box::into_raw(Box::new(
        RegexHandle { regex: re }))
}

// is_match → reuse the compiled regex
#[no_mangle]
pub extern "C" fn is_match(
    h: *mut RegexHandle,
    hay: *const u8, len: usize,
) -> bool {
    let h = unsafe { &*h };
    h.regex.is_match(
        bytes_to_str(hay, len))
}

// destroy → reclaim & drop the Box
#[no_mangle]
pub extern "C" fn destroy(
    h: *mut RegexHandle,
) {
    unsafe { drop(Box::from_raw(h)); }
}
```



borrowed on every call: compiled once, reused

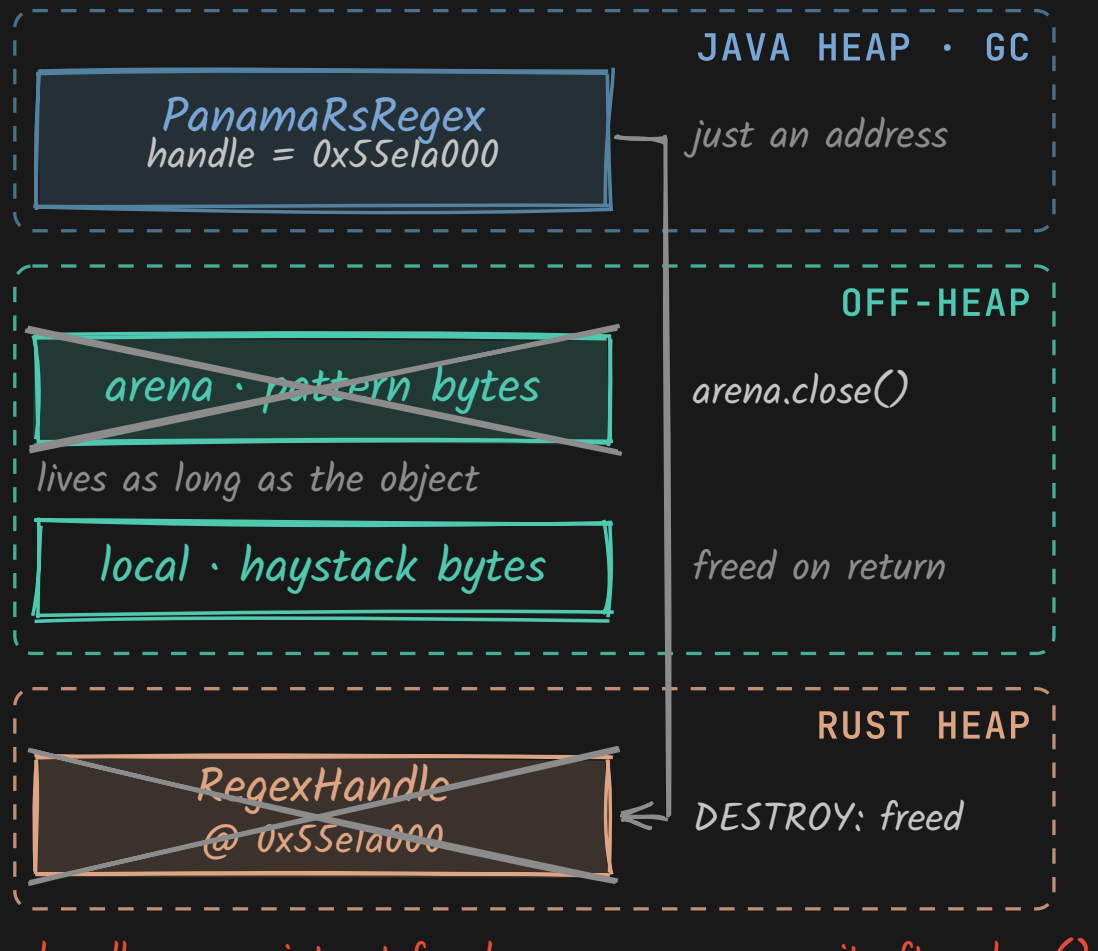
Compiled Regex — Java

```java
class PanamaRsRegex
    implements AutoCloseable {
  final MemorySegment handle;
  final Arena arena =
      Arena.ofConfined();

  // create
  public PanamaRsRegex(String pattern) {
    var cPat = arena.allocateFrom(
        JAVA_BYTE, utf8(pattern));
    handle = (MemorySegment)
        CREATE.invoke(cPat, cPat.byteSize());
  }

  // is_match — scratch arena per call
  boolean isMatch(String hay) {
    try (Arena local = Arena.ofConfined()) {
      var cHay = local.allocateFrom(
          JAVA_BYTE, utf8(hay));
      return (boolean) IS_MATCH.invoke(
          handle, cHay, cHay.byteSize());
    }
  }

  // destroy
  public void close() {
    DESTROY.invoke(handle);
    arena.close();
  }
}
```



Lifetime composition with Arena

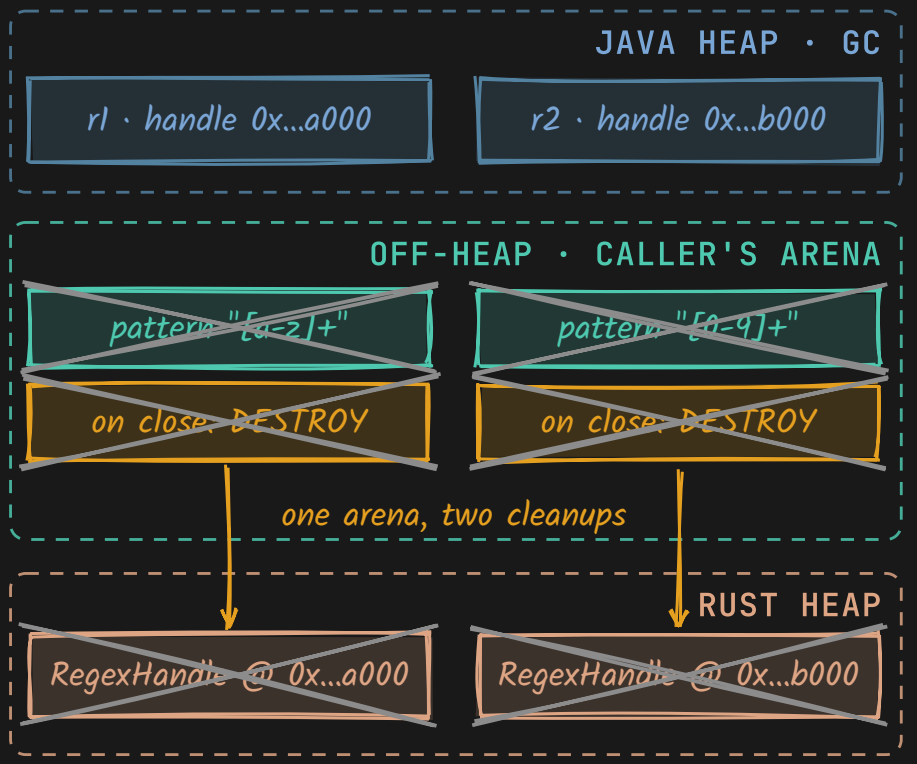
```java
public class PanamaRsRegex {   // no longer AutoCloseable
    private final MemorySegment handle;

    public PanamaRsRegex(String pattern, Arena arena) {
        byte[] utf8 = pattern.getBytes(UTF_8);
        MemorySegment cPat = arena.allocateFrom(JAVA_BYTE, utf8);
        MemorySegment raw = (MemorySegment) CREATE.invoke(
            cPat, (long) utf8.length);
        handle = raw.reinterpret(ADDRESS.byteSize(), arena,
            seg -> DESTROY.invoke(seg));
    }

    public boolean isMatch(String hay) { ... }

    // public void close() { DESTROY.invoke(handle); }
    // not needed: the arena calls DESTROY
}

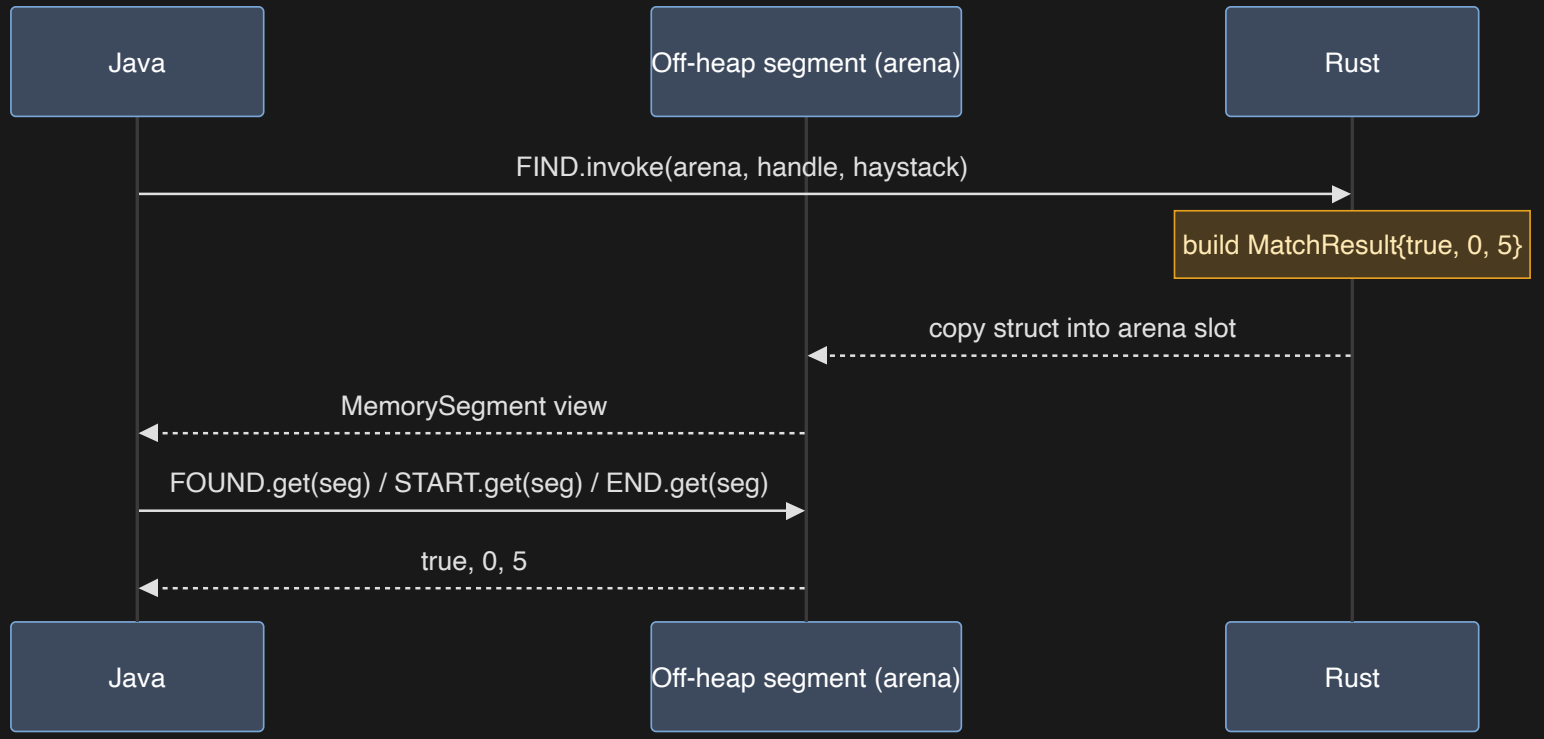
// Usage:
try (Arena arena = Arena.ofConfined()) {
    var r1 = new PanamaRsRegex("[a-z]+", arena);
    var r2 = new PanamaRsRegex("[0-9]+", arena);
    // use r1, r2 ...
}   // arena closes → DESTROY called for both, automatically
```



arena.close() runs every cleanup

one close(), every destructor runs

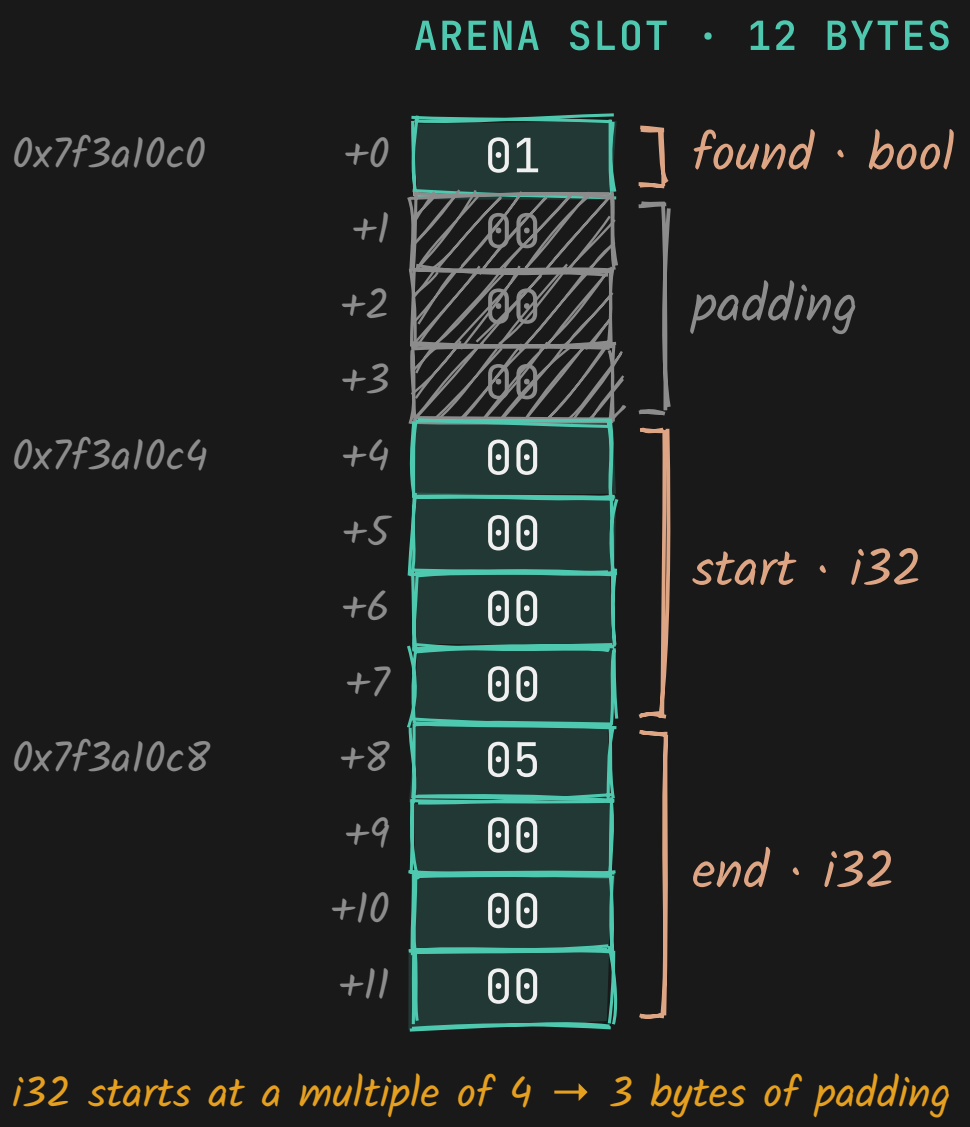
Finding a match



Finding a match — Rust

```rust
#[repr(C)]
pub struct MatchResult {
    pub found: bool,
    pub start: i32,
    pub end:   i32,
}

#[no_mangle]
pub extern "C" fn find(
    handle: *mut RegexHandle,
    hay_ptr: *const u8, hay_len: usize,
) -> MatchResult {
    ...
}
```



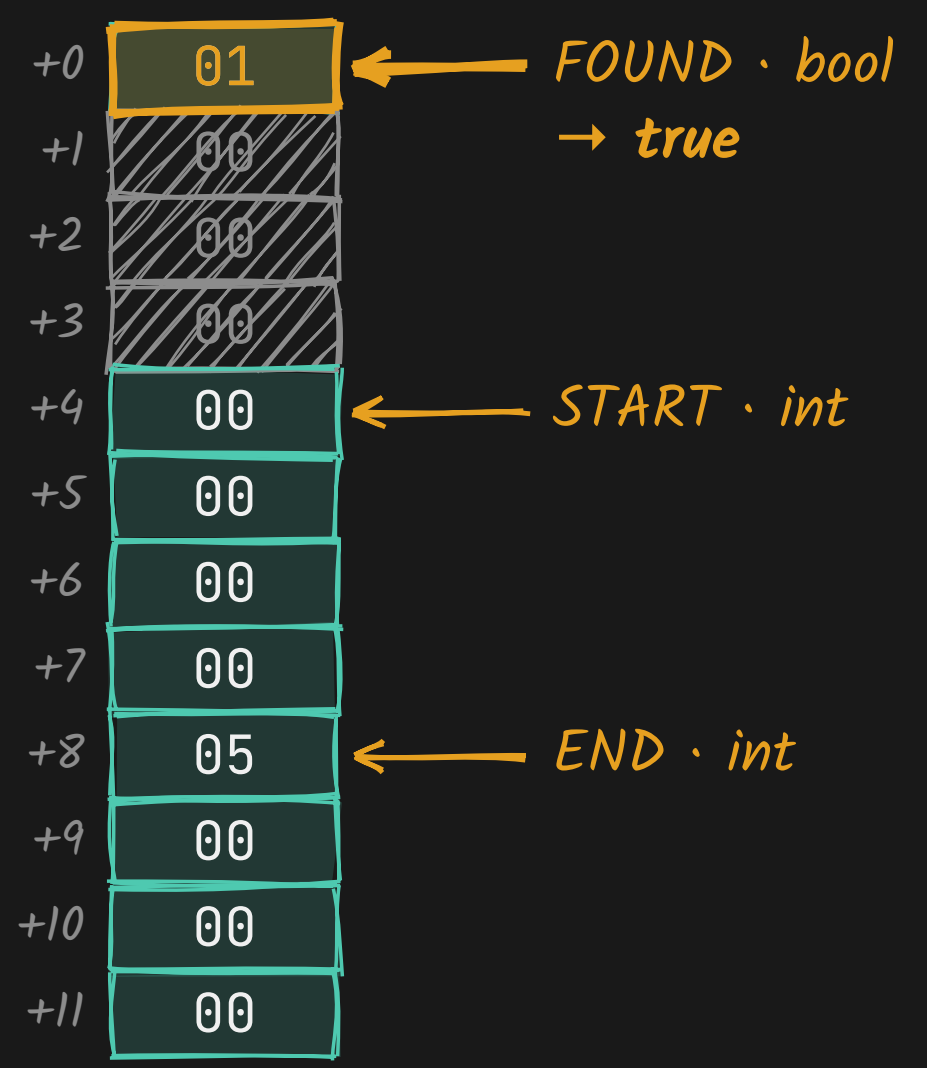
i32 starts at a multiple of 4 → 3 bytes of padding

Finding a match — Java

```java
static final StructLayout MATCH_RESULT = MemoryLayout.structLayout(
  JAVA_BOOLEAN.withName("found"),
  MemoryLayout.paddingLayout(3),   // 3B pad → start/end align to 4
  JAVA_INT.withName("start"),
  JAVA_INT.withName("end"));
static final VarHandle FOUND =
  MATCH_RESULT.varHandle(groupElement("found"));
static final VarHandle START =
  MATCH_RESULT.varHandle(groupElement("start"));
static final VarHandle END   =
  MATCH_RESULT.varHandle(groupElement("end"));

static final MethodHandle FIND =
  Linker.nativeLinker().downcallHandle(...)

public JavaMatchResult find(String haystack) {
  try (Arena arena = Arena.ofConfined()) {
    byte[] utf8 = ...;
    MemorySegment cHay = ...;
    MemorySegment out = (MemorySegment) FIND
      .invoke(
        arena, handle, cHay, cHay.byteSize());
    if (!(boolean) FOUND.get(out, 0L)) {
      // There is no match
    } else {
      // Do something with (int) START.get(out, 0L)
      // and (int) END.get(out, 0L)
    }
  }
}
```



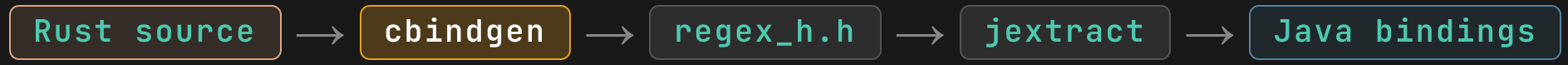
each VarHandle keeps one offset

This is very verbose



```
static final StructLayout MATCH_RESULT = MemoryLayout.structLayout(
  JAVA_BOOLEAN.withName("found"),
  MemoryLayout.paddingLayout(3),   // 3B pad → start/end align to 4
  JAVA_INT.withName("start"),
  JAVA_INT.withName("end"));
static final VarHandle FOUND =
  MATCH_RESULT.varHandle(groupElement("found"));
static final VarHandle START =
  MATCH_RESULT.varHandle(groupElement("start"));
static final VarHandle END   =
  MATCH_RESULT.varHandle(groupElement("end"));
```

From Rust to Java bindings



Generate a C header from Rust

```
cargo install cbindgen
cbindgen --crate jrs-regex-panama --output regex_h.h
```

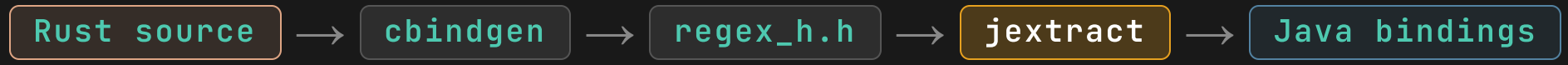
Output

.h

```h
// regex_h.h — generated by cbindgen
struct MatchResult { bool found; int32_t start; int32_t end; };

struct RegexHandle *create(
    const uint8_t *pat_ptr, uintptr_t pat_len);
bool is_match(const struct RegexHandle *handle, ...);
void destroy(struct RegexHandle *handle);
```

From Rust to Java bindings



Auto-generate Java bindings

```
jextract --output src/main/java \
  --target-package org.example.rust.generated \
  regex_h.h
```

```
// Generated by jextract — you don't write this by hand
public static MethodHandle is_match =
    LINKER.downcallHandle(
        LOOKUP.find("is_match").orElseThrow(),
        FunctionDescriptor.of(JAVA_BOOLEAN, ADDRESS, ADDRESS, JAVA_LONG));

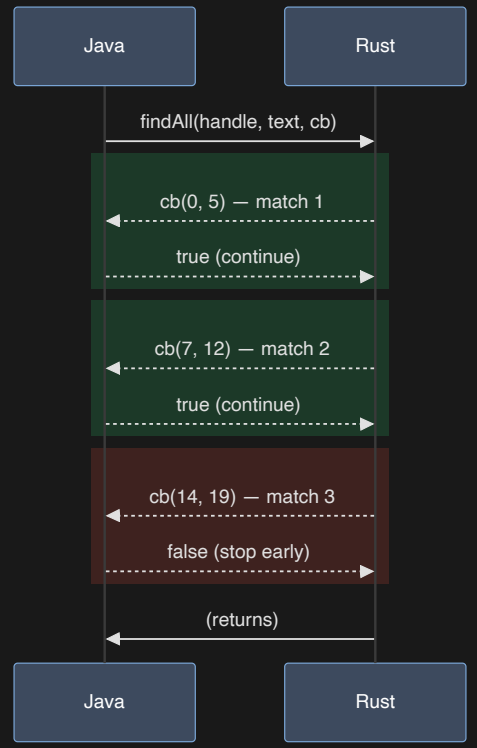
public static StructLayout MatchResult = MemoryLayout.structLayout(
    JAVA_BOOLEAN.withName("found"),
    MemoryLayout.paddingLayout(3),   // jextract gets the padding right
    JAVA_INT.withName("start"),
    JAVA_INT.withName("end"));
```

Can we call Java from Rust?

Callback/upcalls

```java
// Rust calls back into this interface
interface MatchVisitor {
    // true = continue, false = stop early
    boolean onMatch(int start, int end);
}

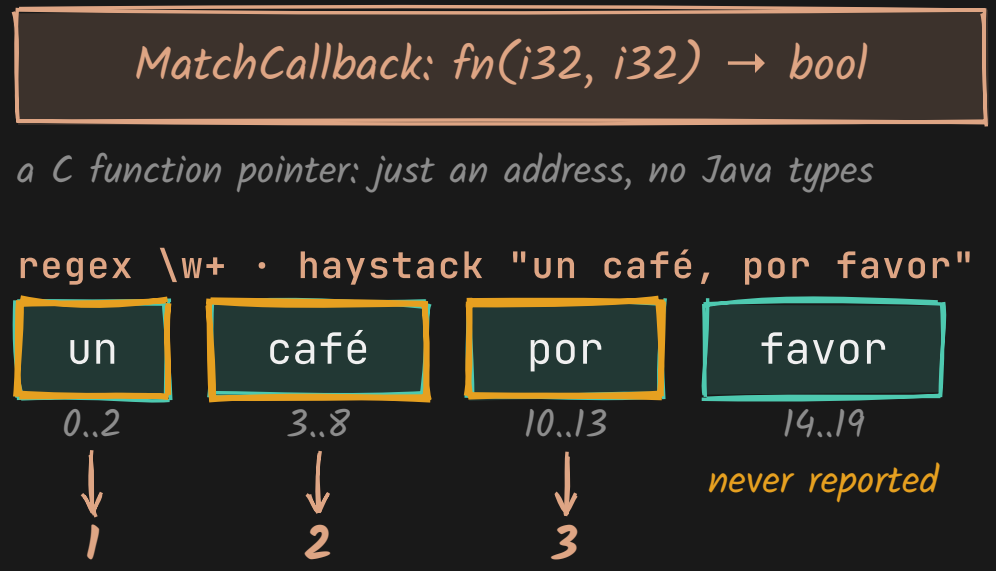
// Usage — pass a lambda
regex.findAll(haystack, (start, end) -> {
    System.out.println(
        "match at " + start + ".." + end);
    return true;  // keep going
});
```



Callbacks — Rust

```rust
// Plain C function pointer — no Java types
pub type MatchCallback =
  unsafe extern "C" fn(
    start: i32, end: i32
  ) → bool;

#[no_mangle]
pub extern "C" fn for_each_match(
    handle: *mut RegexHandle,
    hay_ptr: *const u8, hay_len: usize,
    callback: MatchCallback, // ← the function pointer
) {
  ...
}
```



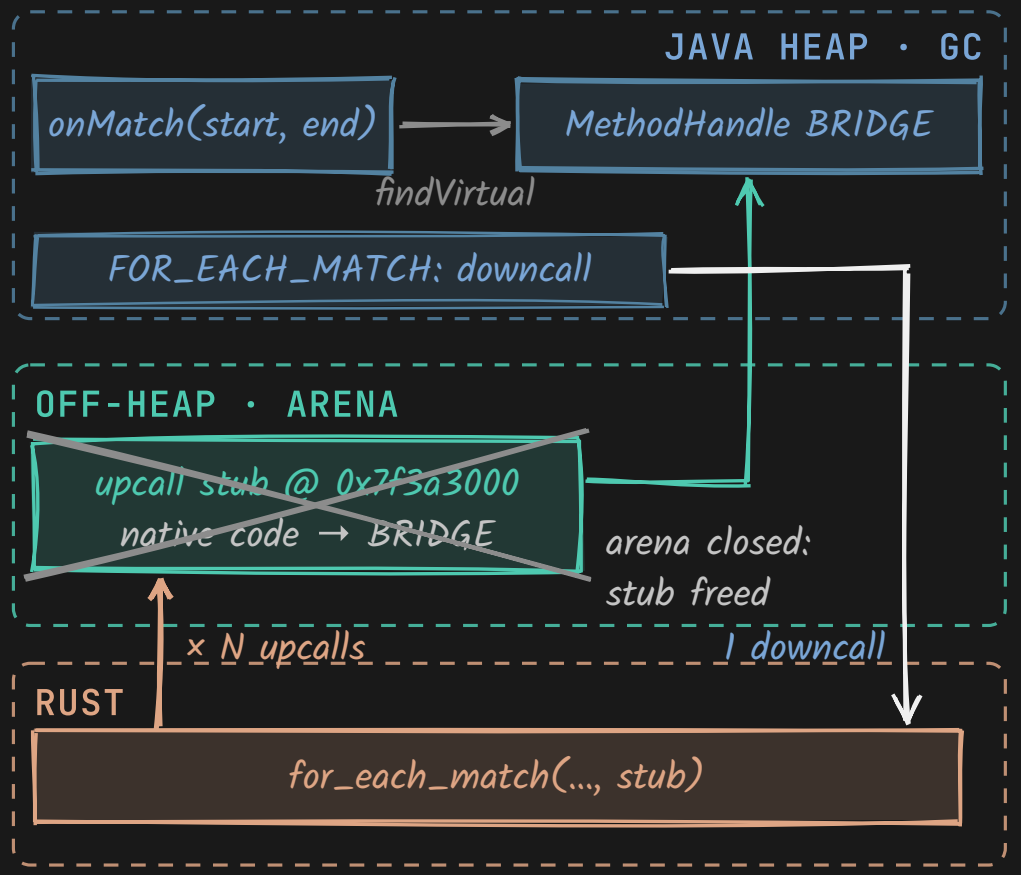
Callbacks — Java

```java
// plain Java method — the callback target
boolean onMatch(int start, int end) { … }

// one-time: lift it into a MethodHandle
static final MethodHandle BRIDGE =
    MethodHandles.lookup().findVirtual(
        PanamaRsRegex.class, "onMatch",
        MethodType.methodType(
            boolean.class, int.class, int.class));

// one-time: downcall (last param = fn pointer)
static final MethodHandle FOR_EACH_MATCH =
  LINKER.downcallHandle(
    LIB.find("for_each_match").orElseThrow(),
    FunctionDescriptor.ofVoid(
        ADDRESS, ADDRESS, JAVA_LONG, ADDRESS));

public void forEachMatch(String haystack) {
  try (Arena arena = Arena.ofConfined()) {
    // a REAL native function pointer
    MemorySegment stub = LINKER.upcallStub(
        BRIDGE.bindTo(this),
        FunctionDescriptor.of(
            JAVA_BOOLEAN, JAVA_INT, JAVA_INT),
        arena);
    MemorySegment cHay = arena.allocateFrom(
        JAVA_BYTE, haystack.getBytes(UTF_8));
    FOR_EACH_MATCH.invoke(
        handle, cHay, cHay.byteSize(), stub);
  }
}
```



CROSSINGS

1 + N 1 downcall, N upcalls

Wrapping up

What we built today

Five shapes that compose into *any* native API

- ✓ Primitives — add(a, b)
- ✓ Strings — isMatch(pattern, haystack)
- ✓ Opaque native handle — a compiled Regex
- ✓ Struct return — find() → {found, start, end}
- ✓ Callback / upcall — streaming findAll

Master these five — the rest of any native API is just composition.

When should you reach for native?

REASON	WORTH IT?	NOTES
Ecosystem	✓ Yes	Now it is easier to use existing libraries
Ergonomics	✓ Yes	Genuine JVM blind spots
Performance	⚠ Verify first	JIT/GC are good. Measure

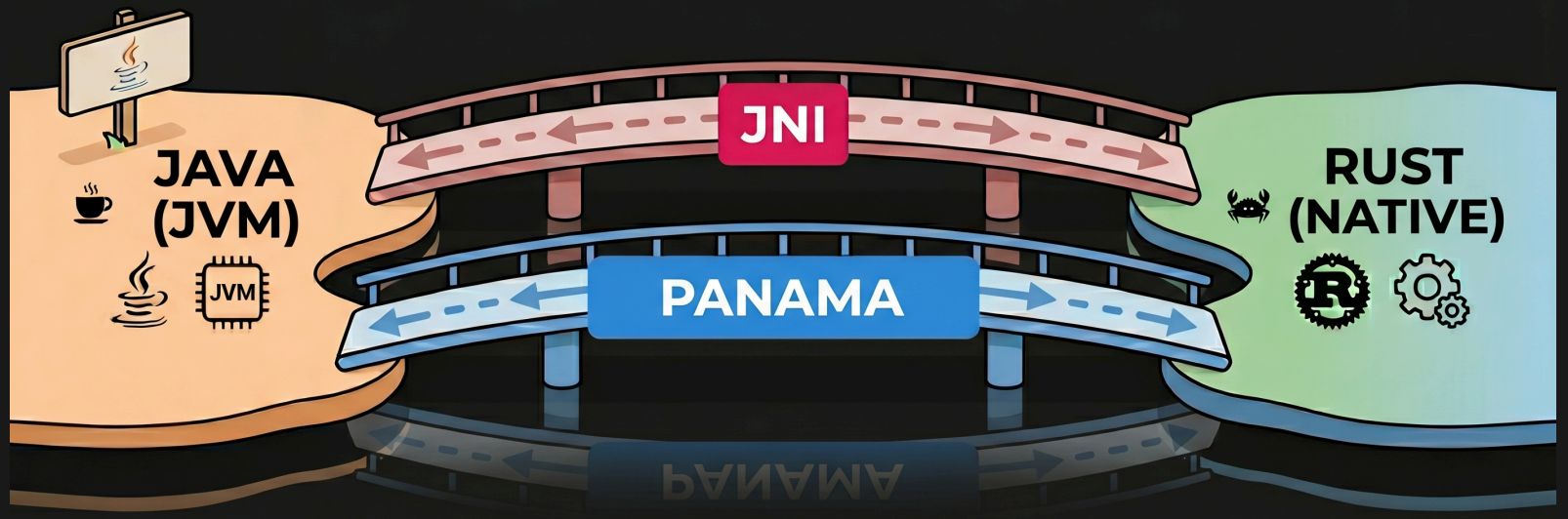
Performance tips

- Each call is expensive.
  - JNI slower than Panama
- One broad > many narrow.
- Shared memory > copies.
- Do not use Java Strings.
- Use async-profiler — not JFR.

Toolbox

- **Build** — `rust-maven-plugin` bundles native libs inside the jar.
- **Debugging** — Two debuggers, two windows.
- **Logging** — Rust `log` / `tracing` don't feed SLF4J.
- **IDEs & AI** — No cross-boundary refactoring. AI shines for translation.
- **Two heaps, one process** — GC knows nothing about Rust allocations.

Same islands, a better bridge



JNI hands native code the JVM · **Panama** hands it plain memory



gortizja.com

Thank you. Questions?